

DAY 06: POSTGRESQL & PSQL

Data Bytes Technologies · Java 25 & Modern Stack

Week 1 gave you the tools. This week we drive real **infrastructure**, and we drive each piece with *its own* command-line tool, from inside the Docker container you started on Day 2. Today that piece is **PostgreSQL**, and the tool is `psql`.

// WHERE WE ARE

- **Day 1–2:** Git, then **Docker & Compose**: one `task up` brings up the course stack (Postgres, Kafka, a Kafka UI) and `docker compose exec` walks you *inside* a running container.
- **Day 3–5:** modern Java and concurrency: shared state, atomicity, "all-or-nothing".

This week we **drive real infrastructure by its own tools**. The Postgres container is already running from Day 2. Today we step inside it with `psql` and speak SQL to it directly.

💡 No Hibernate, no Panache, no Flyway, no Quarkus today. That whole framework layer is **Day 9**. Today is the database *itself* (raw SQL) so you understand what every framework is really doing.

// BY THE END OF TODAY, YOU CAN...

- explain **what a relational database is** and why we reach for PostgreSQL
- open `psql` inside the **Postgres container** and navigate with meta-commands (`\l \dt \d \x \?`)
- write the core SQL: **CREATE TABLE, INSERT, SELECT, UPDATE, DELETE, JOIN**
- pick the right **column types** (incl. `numeric`, `timestamptz`, `uuid`, `jsonb`) and add **constraints** (PK, FK, `NOT NULL`, `UNIQUE`, `CHECK`)
- run a **transaction** (`BEGIN` / `COMMIT` / `ROLLBACK`), read an `EXPLAIN`, query `jsonb`, and take a **backup**

// FIRST: WHAT IS A RELATIONAL DATABASE?

💡 **In plain terms:** a relational database stores data in **tables**: rows and columns, like a spreadsheet with rules. Each table has a fixed set of **typed columns**, and the database **enforces** those rules for you, refusing data that breaks them.

- A **row** is one record (one order); a **column** is one field (its `quantity`).
- "Relational" = tables can **reference** each other: an order points at a customer.
- You ask questions in **SQL**, a declarative language: you say *what* you want, not *how* to fetch it.

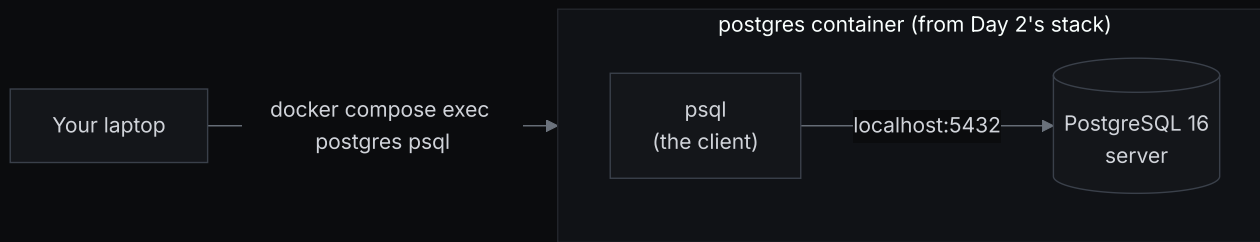
// WHY POSTGRESQL?

- **Open-source, rock-solid, everywhere:** the default serious choice for a new service.
- **Strict where it matters:** real types, constraints, and **ACID transactions** you can trust.
- **Flexible where you need it:** first-class `jsonb` lets one column hold a whole JSON document, relational *and* document, in one engine.
- **Honest standard SQL** with great docs: what you learn here transfers to almost any SQL database.

We pin PostgreSQL 16 in `infra/compose.yaml`, the same version for everyone in the room.

// IT'S ALREADY RUNNING: FROM DAY 2

The Postgres container came up with `task up`. We don't install Postgres on the laptop. We step **inside the container** and run its own client, `psql`:



Because we go *inside*, we never touch the published host port. The client and server are in the same box. Service name, DB, and user all come from `infra/compose.yaml`: `postgres`, `orders`, `orders`.

// TRY IT NOW (2 MIN): OPEN PSQL INSIDE THE CONTAINER

Make sure the stack is up (`task up`), then step inside and open a SQL shell:

```
docker compose -f infra/compose.yaml exec postgres psql -U orders -d orders
```

You should land at the `orders=#` prompt. Ask the server who and where you are:

```
SELECT version();           -- which PostgreSQL is this?
SELECT current_user, current_database();
```

`-U orders` is the **user**, `-d orders` is the **database**. The `=#` prompt means *psql is waiting for SQL*. Type `\q` to leave (but stay in for the next exercise).

// PSQL IS YOUR COCKPIT: META-COMMANDS

`psql` runs **two kinds of input**: **SQL** (ends with `;`) and **meta-commands** (start with `\`), which are psql features, not SQL.

💡 **In plain terms:** backslash commands are psql's own shortcuts for *looking around* and *getting things done* without writing a query. They never leave the shell.

- `\l` list databases · `\dt` list tables · `\d orders` describe a table · `\du` list roles
- `\x` toggle **expanded** rows (wide tables become readable) · `\timing` show how long queries take
- `\e` edit the last query in an editor · `\i file.sql` run SQL from a file · `\copy` import/export CSV
- `\?` meta-command help · `\h CREATE TABLE` SQL syntax help · `\q` quit

// TRY IT NOW (3 MIN): LOOK AROUND

Still at the `orders=#` prompt, explore the empty database:

```
\l           -- the databases on this server
\dt          -- tables in 'orders' (probably "Did not find any relations.")
\du          -- the roles/users (you'll see 'orders')
\timing on   -- now every query reports its duration
\?           -- skim the meta-command help, then press q to exit
```

`\dt` finding nothing is correct. The database is empty. We're about to fix that by building the `orders` table by hand.

// 🖐️ EVERYONE WITH ME?

- you're at an `orders=#` prompt **inside** the Postgres container
- you can tell a **meta-command** (`\dt`) from **SQL** (`SELECT ...;`)
- you ran `\l`, `\dt`, `\du` and read what they returned

If any of that is fuzzy, flag it now. Everything from here is hands-on SQL against this session.

// SQL, IN A FEW VERBS

You'll spend the day in a small, powerful vocabulary. Two families:

- **DDL: define the shape** (the *schema*): `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`.
- **DML: work with the data**: `INSERT`, `SELECT`, `UPDATE`, `DELETE`.

💡 **In plain terms:** first you *define* a table once (DDL), then you *use* it many times (DML). A **schema** is just the shape you promise your data will have, and Postgres enforces that promise.

// CHOOSING COLUMN TYPES: PICK THE RIGHT BOX

The type is the first constraint. Choose for what the value *is*, not what's convenient:

- `text`: strings. (In Postgres `text` and `varchar(n)` perform identically; `varchar(n)` only adds a length cap.)
- `integer` / `bigint`: whole numbers. Identity keys are usually `bigint`.
- `numeric(p, s)`: exact decimals for **money**. *Never* `float` / `real` for money. They round.
- `timestamptz`: an instant in time, stored in UTC. Prefer it over `timestamp` (no zone).
- `boolean`, `uuid` (`gen_random_uuid()`), and `jsonb`: a whole JSON document in a column.

// WHY `jsonb` ? SCHEMA FOR THE UNKNOWNABLE

Some data has no fixed shape: a partner's extra fields, feature flags, sparse tags. A column each means a schema change every time a new one appears.

💡 **In plain terms:** `jsonb` is a column that holds a whole JSON document. Postgres parses and stores it efficiently, lets you **query inside it**, and can even **index inside it** (GIN indexes).

The rule: `jsonb` for **sparse, caller-defined** data. **Not** for anything you filter, join, or constrain heavily: those earn a real, typed column. `jsonb` is not an excuse to skip schema design.

// DESIGNING THE ORDERS TABLE

Most fields are ordinary typed columns; one (`attributes`) is deliberately open-ended:

ORDERS		
bigint	id	PK
text	customer_id	
text	sku	
int	quantity	
numeric	unit_price	
text	status	
jsonb	attributes	
timestamptz	created_at	
timestamptz	updated_at	

// TRY IT NOW (4 MIN): CREATE TABLE BY HAND

Type this at the `orders=#` prompt (the constraints come next slide. For now, build the shape):

```
CREATE TABLE orders (  
  id          bigint          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  customer_id text            NOT NULL,  
  sku         text            NOT NULL,  
  quantity    integer         NOT NULL CHECK (quantity > 0),  
  unit_price   numeric(12,2)  NOT NULL CHECK (unit_price >= 0),  
  status       text            NOT NULL DEFAULT 'NEW'  
                                CHECK (status IN ('NEW', 'PAID', 'SHIPPED', 'CANCELLED')),  
  attributes   jsonb           NOT NULL DEFAULT '{} '::jsonb,  
  created_at   timestamptz    NOT NULL DEFAULT now(),  
  updated_at   timestamptz    NOT NULL DEFAULT now()  
);
```

Then `\d orders`, read the columns, types, defaults, and constraints back.

// CONSTRAINTS: THE DATABASE GUARDS THE VAULT

Your app validates input; the **database** enforces invariants no matter who writes. Defence in depth.

- **PRIMARY KEY**: a unique, non-null identity for each row (our **id**).
- **NOT NULL**: this column must always have a value.
- **UNIQUE**: no two rows share this value (e.g. a customer's email).
- **CHECK (...)**: a rule the value must satisfy (**quantity > 0** , **status IN (...)**).
- **FOREIGN KEY ... REFERENCES**: this value must exist in another table (next section).

// TRY IT NOW (4 MIN): INSERT, AND WATCH A CHECK BITE

Add some rows, then try to break a rule on purpose:

```
INSERT INTO orders (customer_id, sku, quantity, unit_price, status) VALUES
('alice', 'SKU-1', 2, 9.99, 'NEW'),
('alice', 'SKU-2', 1, 49.50, 'PAID'),
('bob', 'SKU-1', 5, 9.99, 'SHIPPED');

SELECT * FROM orders;           -- note the auto-filled id, created_at, attributes

INSERT INTO orders (customer_id, sku, quantity, unit_price)
VALUES ('eve', 'SKU-3', 0, 1.00); -- quantity 0 violates CHECK (quantity > 0)
```

That last `INSERT` is rejected: `new row violates check constraint`. The database refused bad data so your code never has to find it later. That's the whole point.

// SELECT: ASK QUESTIONS OF YOUR DATA

SELECT is where you'll live. Shape the answer with **WHERE**, **ORDER BY**, and **LIMIT**:

```
SELECT id, sku, quantity, unit_price      -- which columns
FROM orders                               -- from which table
WHERE quantity >= 2 AND status <> 'CANCELLED' -- which rows
ORDER BY unit_price DESC                   -- in what order
LIMIT 10;                                -- how many
```

Read it as a sentence: *which columns, from which table, which rows, in what order, how many*. ***** means "all columns", handy at the prompt, but name columns in real queries.

// TRY IT NOW (3 MIN): QUERY THE ORDERS

```
SELECT customer_id, sku, unit_price FROM orders ORDER BY unit_price DESC;  
  
SELECT * FROM orders WHERE customer_id = 'alice';  
  
SELECT count(*) FROM orders WHERE status = 'NEW';  
  
SELECT sku, quantity FROM orders WHERE quantity > 1 ORDER BY quantity DESC LIMIT 2;
```

Try changing the `WHERE` and `ORDER BY`. The same table answers very different questions.

// UPDATE & DELETE: NEVER WITHOUT A WHERE

Change and remove rows with the same clauses you already know:

```
UPDATE orders SET status = 'PAID' WHERE id = 1;      -- one row
SELECT id, status, updated_at FROM orders WHERE id = 1;

DELETE FROM orders WHERE status = 'CANCELLED';      -- only cancelled rows
```

💡 In plain terms: `UPDATE` and `DELETE` change **every row that matches the** `WHERE`. Forget the `WHERE` and you change **all of them**. Always `SELECT` your `WHERE` first to see what it hits.

// RELATING TABLES: FOREIGN KEYS

Right now `customer_id` is just text. Nothing stops a typo. Let's make it **point at a real customer**. First a `customers` table, then a foreign key from `orders` to it:

// TRY IT NOW (4 MIN): ADD CUSTOMERS AND A FOREIGN KEY

```
CREATE TABLE customers (  
  id          text PRIMARY KEY,  
  name        text NOT NULL,  
  email       text UNIQUE NOT NULL,  
  created_at  timestampz NOT NULL DEFAULT now()  
);  
  
INSERT INTO customers (id, name, email) VALUES  
  ('alice', 'Alice Adams', 'alice@example.com'),  
  ('bob',   'Bob Brown',   'bob@example.com');  
  
-- Now tie orders to customers. This SUCCEEDS only because every existing  
-- order's customer_id ('alice','bob') already exists in customers:  
ALTER TABLE orders  
  ADD CONSTRAINT orders_customer_fk FOREIGN KEY (customer_id) REFERENCES customers (id);
```

// JOIN AND AGGREGATE ACROSS TABLES

A `JOIN` stitches rows from two tables together on a matching column:

```
SELECT o.id, c.name, o.sku, o.quantity, o.unit_price
FROM orders o
JOIN customers c ON c.id = o.customer_id
ORDER BY o.id;
```

- `o` and `c` are **aliases**; `ON c.id = o.customer_id` glues each order to *its* customer, so one query shows the customer's **name**, not just an id.
- **Aggregates** (`count`, `sum`, `avg`, `min`, `max`) collapse many rows into one summary number.
- `GROUP BY` gives one summary *per group*, and every selected column must be in the `GROUP BY` or inside an aggregate. (You'll write a per-customer total next.)

// TRY IT NOW (3 MIN): JOIN AND SUMMARISE

```
-- every order with its customer's name:
SELECT o.id, c.name, o.sku, o.unit_price
FROM orders o JOIN customers c ON c.id = o.customer_id
ORDER BY o.id;

-- spend per customer:
SELECT c.name, count(*) AS orders, sum(o.quantity * o.unit_price) AS total
FROM orders o JOIN customers c ON c.id = o.customer_id
GROUP BY c.name ORDER BY total DESC;
```

Two queries, two very different questions, from the same two tables, joined on one column.

// INDEXES: WHY A QUERY GETS SLOW

Without an index, Postgres reads **every row** to find matches, a *sequential scan*. Fine for 3 rows; painful for 3 million. An **index** is a sorted lookup structure that finds rows fast.

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 'alice'; -- how will Postgres run this?
```

💡 **In plain terms:** `EXPLAIN` shows the **plan**: how Postgres intends to find your rows (a full scan, or an index). It's how you see *why* a query is slow before you guess.

// TRY IT NOW (3 MIN): READ A PLAN, ADD AN INDEX

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 'alice'; -- likely "Seq Scan"

CREATE INDEX orders_customer_idx ON orders (customer_id); -- build the index

EXPLAIN SELECT * FROM orders WHERE customer_id = 'alice'; -- the plan can now use it

\d orders -- the index now appears under "Indexes:"
```

On three rows the planner may *still* choose a scan. That's correct (scanning 3 rows beats a lookup). The skill is **reading the plan**; the payoff arrives at scale.

// TRANSACTIONS: ALL-OR-NOTHING

Sometimes several statements must **all** succeed, or **none** of them should. That's a transaction, and it's the database's answer to the shared-state problem from concurrency week.

💡 In plain terms: `BEGIN` opens a transaction; your changes are **invisible to everyone else** and not yet permanent. `COMMIT` makes them all durable at once; `ROLLBACK` throws them **all** away, as if nothing happened.

This is the **A** in ACID, *atomicity*. And **isolation** (the *I*) means concurrent transactions don't see each other's half-done work. Postgres's default keeps your reads consistent.

// A TRANSACTION, START TO FINISH

// TRY IT NOW (4 MIN): ROLLBACK RESCUES YOU

Make a change inside a transaction, see it, then undo it completely:

```
BEGIN;
UPDATE orders SET status = 'CANCELLED';      -- oops, no WHERE, ALL rows!
SELECT id, status FROM orders;                -- everything is CANCELLED... in this session

ROLLBACK;                                     -- undo the whole transaction
SELECT id, status FROM orders;                -- back to how it was, nothing was saved
```

The reckless `UPDATE` from two slides ago: survivable, because it was inside a transaction. `ROLLBACK` is your SQL undo. Swap it for `COMMIT` only when you're sure.

// QUERYING INSIDE JSONB

The `attributes` column holds JSON. Three operators do most of the work:

```
SELECT id, attributes -> 'channel' FROM orders; -- -> returns JSON
SELECT id, attributes ->> 'channel' FROM orders; -- ->> returns TEXT
SELECT id FROM orders WHERE attributes @> '{"gift": true}'; -- @> "contains"
```

`->` digs in and returns **JSON**; `->>` returns the value as **text** (use this in `WHERE`); `@>` asks "does this document contain that?", and `@>` can use a GIN index for speed.

// TRY IT NOW (3 MIN): STORE AND QUERY JSON

```
INSERT INTO orders (customer_id, sku, quantity, unit_price, attributes)
VALUES ('alice', 'SKU-7', 1, 49.99, '{"channel":"web","gift":true}');

-- only the gift orders, showing their channel as text:
SELECT id, attributes ->> 'channel' AS channel
FROM orders
WHERE attributes ->> 'gift' = 'true';

-- containment: orders placed via web
SELECT id FROM orders WHERE attributes @> '{"channel":"web"}';
```

Same table, now answering questions about **schemaless** data, without a single new column.

// BACKUPS: YOUR DATA, PORTABLE

A database is only as safe as its last backup. `pg_dump` is another tool that lives **inside the container**, same Day 2 move:

```
# from your laptop; -T sends no TTY so the redirect captures clean SQL:
docker compose -f infra/compose.yaml exec -T postgres \
  pg_dump -U orders orders > orders-backup.sql
```

The dump is **plain SQL**: every `CREATE TABLE` and `INSERT` to rebuild the database. Restore by feeding it back: `psql`
`-U orders -d orders -f orders-backup.sql` (or `\i` inside psql).

// COMMON TRAPS

- `UPDATE` / `DELETE` with no `WHERE`: you change every row. `SELECT` the `WHERE` first; wrap risky work in `BEGIN ... ROLLBACK`.
- `float` for money: it rounds. Use `numeric(p, s)`. And prefer `timestampz` over `timestamp`.
- Everything in `jsonb`: anything you filter, join, or constrain wants a real, typed column.
- "double quotes" for strings: those are *identifiers*; string literals use `'single quotes'`.
- Forgetting the `;`: the prompt sits on `orders-#` waiting; just type `;` and Enter.

// RECAP, IN PLAIN TERMS

Today you spoke to a real database, by hand:

- you opened `psql` inside the container and navigated with meta-commands (`\l \dt \d \x`)
- you **designed and built** the `orders` and `customers` tables with the right types and constraints
- you wrote the core SQL (`INSERT`, `SELECT`, `UPDATE`, `DELETE`, `JOIN`, `GROUP BY`) and let `CHECK` and `FOREIGN KEY` reject bad data
- you read an `EXPLAIN`, ran a **transaction** (`BEGIN/COMMIT/ROLLBACK`), queried `jsonb`, and took a **backup**

No framework did any of this for you, which means when one does, on Day 9, you'll know exactly what it's typing.

// TODAY'S LAB

 **Build a small order database by hand:** connect with `psql` inside the container, create the `customers` and `orders` tables with constraints and a foreign key, load data, and answer real questions with `SELECT`, `JOIN`, and `GROUP BY`. Then read an `EXPLAIN`, prove a `ROLLBACK`, query `jsonb`, and take a `pg_dump`.

Brief: `labs/day-06/` · You practise on the **real container**, no starter scaffold.

Acceptance: tables created with constraints, a `CHECK` and a `FOREIGN KEY` you saw reject bad data, a `JOIN` + `GROUP BY` answering a real question, a `ROLLBACK` that undid a change, and a backup file on disk.

// TAKEAWAY

A database isn't a black box behind a framework. It's a server you can **talk to directly**: design typed tables, let **constraints** guard the data, ask questions in **SQL**, and make change safe with **transactions**. You drove it with its own tool, `psql`, from inside the container.

Tomorrow, Day 7: Kafka. Another real service in the same stack, driven the same way: its **CLI, from inside its Docker container**. Different tool, identical move: step in, and speak to it directly.